Asynchronous, Generation-Free ABC: Streaming AMIS Reweighting for Heterogeneous Simulator Workloads on HPC

First Author[1*] and Second Author[1]

[1*]Department, Organization, City, Country.

*Corresponding author(s). E-mail(s): first.author@example.com;
Contributing authors: second.author@example.com;

Abstract

Sequential approximate Bayesian computation (ABC-SMC/PMC) is the standard tool for likelihood-free inference, but its generation-staged structure imposes synchronization barriers that waste compute when simulator runtimes vary across parameters, precisely the regime of large heterogeneous high-performance computing (HPC) workloads. We introduce a *generation-free, single-arrival-driven* ABC algorithm that combines smooth-kernel ABC with a streaming limit of Adaptive Multiple Importance Sampling (AMIS): the proposal mixture and importance weights update after *every* evaluated particle rather than at population boundaries. The algorithm is *stateless* (a pure function of the evaluated history), which makes it crash-recoverable and a natural fit for the barrier-free island model of the Propulate optimization engine. We show that the *idealized* streaming estimator inherits AMIS's consistency and central-limit guarantees under the Wilkinson smooth-likelihood framework. For efficient execution we additionally provide an optional sliding-buffer, top-$\boldsymbol{k}$ approximation of the estimator; it lies outside the scope of those guarantees, so rather than claim the asymptotics for it we supply direct empirical evidence, via simulation-based calibration, that it remains well calibrated. Empirically, against a synchronous pyABC baseline matched on the ABC kernel and bandwidth schedule (so the systems comparison turns on synchronization rather than the acceptance rule), the method (i) is well calibrated: simulation-based calibration on a model with analytic posterior yields empirical coverage close to nominal; and (ii) delivers substantial HPC benefits: under a persistent straggler the synchronous baseline's throughput collapses by more than two orders of magnitude while the asynchronous method holds essentially flat, and under runtime heterogeneity it sustains several-fold higher simulation throughput at a fixed wall-clock budget.

On a realistic Cellular Potts tissue-simulation workload the asynchronous method scales almost linearly to eight nodes, sustaining several-fold higher throughput ($\approx$ **5**$\times$ at **384** workers, widening with scale) than a synchronous baseline that, given the same core budget and a matching population, scales only sub-linearly, at comparable posterior quality.

Keywords: approximate Bayesian computation, sequential Monte Carlo, adaptive multiple importance sampling, asynchronous algorithms, high-performance computing, likelihood-free inference

1 Introduction

Approximate Bayesian computation (ABC) is a standard tool for simulator-based inference when the likelihood is unavailable or too expensive to evaluate directly [1]. Sequential ABC methods improve efficiency by evolving proposal distributions and tolerance thresholds across a sequence of intermediate targets [2–5]. In practice, however, most ABC-SMC/PMC implementations remain organized around *synchronous* populations: the proposal and tolerance update only after an entire generation has been evaluated. Because simulator runtimes often depend strongly on the sampled parameters, generation barriers leave compute nodes idle waiting for the slowest simulation, a dominant inefficiency on large heterogeneous high-performance computing (HPC) systems.

This paper introduces a generation-free alternative designed for exactly that regime and implemented within the barrier-free island model of the Propulate engine [6]. Our contributions are:

1. A **generation-free, single-arrival-driven ABC algorithm** that combines smooth-kernel ABC [7, 8] with a streaming limit of Adaptive Multiple Importance Sampling (AMIS) [9]. To our knowledge it is the first ABC algorithm whose proposal mixture updates after every evaluated particle rather than at generation/stage boundaries.
2. A **history-reconstructed particle archive** that makes the algorithm *stateless* (a pure function of the evaluated history) and therefore crash-recoverable.
3. **Integration into Propulate**, exploiting its barrier-free island model for true asynchronous execution on HPC.
4. **Asymptotic guarantees for the estimator, with empirical validation of an efficient variant.** We give consistency and a central limit theorem for the *idealized* full-mixture streaming estimator, as a corollary of AMIS theory under the smooth-likelihood framework and with explicit conditions on the proposal sequence and bandwidth schedule (§4). For efficient execution we additionally provide an optional sliding-buffer, top-k approximation of this estimator; it lies outside the scope of those conditions, so we establish its inference quality *empirically*, by simulation-based calibration and matched-budget posterior error, rather than by appeal to the asymptotics.

5. An **apples-to-apples empirical evaluation against pyABC** [10, 11] in which pyABC's acceptor is replaced by a probabilistic-rejection acceptor using the *same* ABC kernel and bandwidth schedule. This matched-kernel baseline lets us attribute the throughput and utilization gains to barrier removal, while we separately verify (via SBC and matched-budget posterior error) that the streaming estimator's inference quality is comparable to the synchronous baseline's.

The remainder of the paper summarizes the relevant ABC and importance-sampling background (§2), develops the method (§3) and its theory (§4), describes the implementation (§5), lays out the experimental program (§6), and reports results (§7).

2 Background and Related Work

2.1 Approximate Bayesian computation

ABC replaces exact likelihood evaluation with simulation and discrepancy-based acceptance. For observed data y and parameter θ, the ABC target at bandwidth ϵ is

$$\pi_\epsilon(\theta \mid y) \propto \pi(\theta)\, L_\epsilon(\theta), \qquad L_\epsilon(\theta) = \mathbb{E}_{\rho \sim p(\rho \mid \theta)}[K_\epsilon(\rho)]\,, \tag{1}$$

where ρ is the discrepancy between simulated and observed summaries, π is the prior, and K_ϵ is the ABC kernel. Classical ABC uses the hard indicator $K_\epsilon(\rho) = \mathbf{1}[\rho < \epsilon]$; *smooth-kernel* ABC [7, 8] replaces this with a continuous kernel (e.g. Gaussian $\exp(-\rho^2/2\epsilon^2)$ or Epanechnikov), which removes the discontinuity in the likelihood approximation and admits standard SMC-sampler theory.

2.2 Sequential ABC algorithms

The generation-staged family (ABC-SMC [2, 3], ABC-PMC [4], and adaptive variants [5, 12, 13]) advances through intermediate targets using mixture proposals

$$q_r(\theta) = \sum_{j=1}^{N_r} W_j^{(r)}\, K(\theta \mid \theta_j^{(r)}) \tag{2}$$

built from accepted particles of the previous stage. The defining constraint is the *generation barrier*: the importance weights at stage r are computed against a single proposal q_{r-1}, which requires the full stage-r population before q_r can be formed.

2.3 Adaptive multiple importance sampling

AMIS [9] generalizes staged importance sampling: at each stage the proposal is adapted from past particles, and crucially *all* past particles are re-weighted against the cumulative proposal mixture $\frac{1}{n}\sum_{j=1}^{n} q_{\tau_j}$ (the balance heuristic of [14]) rather than against the proposal under which each was originally drawn. AMIS is consistent and satisfies a CLT under regularity conditions. Our method takes the *streaming limit* of AMIS: stage size one, single-arrival-driven adaptation.

2.4 Asynchronous SMC and parallel ABC

Off-barrier SMC has been studied for state-space models (asynchronous anytime particle methods [15] and parallel resampling [16]), but none targets ABC, and none combines asynchronous execution with AMIS-style cumulative-mixture reweighting. Distributed ABC frameworks such as pyABC [10, 11] and ABCpy [17] parallelize *simulation* effectively but retain population-level synchronization: workers stall while in-flight simulations drain at the generation boundary. Our contribution removes the barrier entirely; the trade-off is that standard generational SMC theory no longer applies, which motivates §4.

3 Method

3.1 Design constraints and history-based state

We design the algorithm to be *stateless*: every quantity that defines the reported estimator and the archive is a pure function of the evaluated history. Statelessness is a desirable property in its own right, as it makes the reported estimator reproducible up to MPI arrival order and crash-recoverable, each candidate being reconstructible from the recorded history alone; the only carried state — the online AMIS snapshot buffer and two scheduler throttles — is performance state, rebuilt empty after a restart, which the retroactive estimator supersedes. For an efficient implementation we leverage the existing Propulate framework, whose propagator exposes a single call `__call__(inds) -> Individual`: it receives the entire evaluated history and emits the next candidate without persistent archive state or assimilation callbacks, exactly the stateless, history-based interface our design calls for. All algorithmic state is thus reconstructed from history. Define the evaluated history at call n as

$$\mathcal{H}_n = \left\{(\theta_i, \rho_i, \epsilon_i, \tau_i, w_i)\right\}_{i=1}^{n}, \tag{3}$$

where θ_i is the parameter, ρ_i the discrepancy, ϵ_i the bandwidth active when θ_i was proposed, τ_i the proposal-time index, and w_i the stored core importance weight π/q_{τ_i}.

3.2 Tolerance reconstruction and monotonicity

Each proposed individual stores the bandwidth active at proposal time. The effective bandwidth reconstructed from history is the running minimum of these stored bandwidths, $\epsilon_{\text{hist}} = \min_i \epsilon_i$ (falling back to the initial tolerance on an empty history). The scheduler proposes ϵ_{sched}, and the bandwidth used is $\epsilon_n = \min(\epsilon_{\text{hist}}, \epsilon_{\text{sched}})$, giving a monotone-decrease guarantee with no mutable propagator state.

3.3 Archive and smooth-kernel proposal

The reconstructed archive is the top-k history members by lowest discrepancy, $A_n = \mathrm{Top}_k(\{\theta_i\}, \text{order by } \rho_i)$. The proposal mixture uses kernel-weighted archive members,

$$q_n(\theta) = \sum_{j \in A_n} \tilde{W}_j^{(n)} K_\Sigma(\theta - \theta_j), \qquad \tilde{W}_j^{(n)} \propto w_j \cdot K_{\epsilon_n}(\rho_j), \tag{4}$$

with $\sum_j \tilde{W}_j^{(n)} = 1$. The perturbation kernel K_Σ is Gaussian with covariance $\Sigma_n = s\,\widehat{\mathrm{Cov}}_{\tilde{W}}(A_n)$ estimated from the kernel-weighted archive (s is the `perturbation_scale` hyperparameter), factored once via Cholesky. Smooth kernels remove the prior-vs-archive discontinuity present in hard-threshold ABC-PMC: every evaluated particle contributes continuously through its kernel weight.

3.4 Streaming AMIS importance weight

A particle $\theta^\star$ drawn from q_n receives an importance weight under the balance heuristic over a sliding ring buffer $\mathcal{S}$ of past proposals (size S, sampled every `amis_interval` calls):

$$w^\star = \frac{\pi(\theta^\star)}{\bar{q}_n(\theta^\star)}, \qquad \bar{q}_n(\theta) = \frac{1}{1 + |\mathcal{S}|}\Big[q_n(\theta) + \sum_{s \in \mathcal{S}} q_s(\theta)\Big]. \tag{5}$$

Setting $|\mathcal{S}| = 0$ recovers single-current-proposal weighting; increasing S enriches the balance-heuristic denominator toward the full cumulative mixture, which lowers the variance of the importance weights at a modest additional per-call cost. We use $S = 20$ by default, a value that captures most of that variance reduction while keeping each update cheap. A particle's weight, assigned at proposal time, is corrected by the cumulative-mixture denominator at every subsequent use (the retroactive AMIS step). Because each snapshot q_s is the proposal reconstructed from the history truncated at call s, the buffer $\mathcal{S}$ is itself a deterministic function of $\mathcal{H}_n$, so the propagator stores nothing between calls. Equation (5) is the proposal-time importance weight; the *posterior* estimator reported in §4 reweights it by the smooth ABC likelihood $K_{\epsilon_n}(\rho)$ (6).

Three weight objects.

To avoid conflation, the method uses three distinct weightings. (i) The *adaptation weights* $\tilde{W}_j^{(n)}$ (4) are kernel weights over the top-k archive; they define the next proposal mixture q_n and so drive proposal *adaptation* only. (ii) The *proposal-time importance weight* $w^\star$ (5) is the balance-heuristic weight over the bounded sliding buffer $\mathcal{S}$ ($S = 20$), assigned online; it enters proposal adaptation through (4) and the effective-sample-size diagnostic, but never the reported posterior. (iii) The *reported posterior* is the retroactive estimator $\widehat{\pi}_n$ (6), in which every evaluated particle is reweighted by $\pi(\theta_i)K_{\epsilon_n}(\rho_i)/\bar{q}_n(\theta_i)$, where $\bar{q}_n$ is a draw-proportional mixture of $m \le S = 20$ history-reconstructed proposals plus a defensive prior component (Appendix A) — the Condition 4 approximation to the full cumulative mixture, not the mixture over every evaluated particle. The reported posterior densities and the

SBC calibration are computed from it; the Wasserstein-vs-time quality curves are computed on the top-k archive (asynchronous) and the final population (synchronous), matched in size.

Stored fields and weight lifecycle.

Each evaluated individual i records exactly four fields: its parameter θ_i, its discrepancy ρ_i, the bandwidth ϵ_{τ_i} active when it was proposed, and its proposal-time weight $w_i^\star$ (5); the history $\mathcal{H}_n$ is the arrival-ordered sequence of these records. From $\mathcal{H}_n$ alone: the archive is the top-k members by smallest ρ under the reconstructed running-minimum bandwidth; the adaptation weights $\tilde{W}_j^{(n)}$ (weight (i)) are *recomputed* each call from the current archive (never stored) and define q_n; the proposal-time weight $w_i^\star$ (weight (ii)) is computed once at proposal time over the sliding buffer $\mathcal{S}$, stored, and consumed by the ESS diagnostic and by parent selection during adaptation; and the reported posterior weight (weight (iii)) is *recomputed* retroactively from $\mathcal{H}_n$ using the $m \leq S$ snapshot denominator with a defensive prior floor. Only $w_i^\star$ is persisted; weights (i) and (iii) are pure functions of $\mathcal{H}_n$ recomputed on demand, so after a crash the archive and the reported estimator are reconstructed from the recorded history, while the online buffer and the two scheduler throttles rebuild empty and affect only the proposal-time diagnostic.

3.5 Update loop

Algorithm 1 assembles the pieces above into the single-arrival update the propagator runs on each call. Every quantity it uses (the effective bandwidth, the archive, the proposal mixture, and the AMIS denominator) is reconstructed from the history $\mathcal{H}_n$ passed in, so the update carries no state of its own between calls; until the history holds at least k members it emits uniform prior draws to seed the archive.

Algorithm 1 History-reconstructed, single-arrival-driven ABC update

Require: History $\mathcal{H}_n$, archive size k, initial bandwidth ϵ_0, snapshot buffer $\mathcal{S}$

Reconstruct ϵ_{hist} from stored tolerances; compute ϵ_{sched}; set $\epsilon_n \leftarrow \min(\epsilon_{\text{hist}}, \epsilon_{\text{sched}})$
if $|\mathcal{H}_n| < k$ **then**
    emit a uniform prior draw (bootstrap) ▷ covers $\bar{q}$ on the whole box
else
    select archive A_n (top-k by ρ); form $\tilde{W}^{(n)}$ in log-space via K_{ϵ_n}
    build Σ_n, Cholesky-factor once; sample $\theta^\star$ by perturbing a $\tilde{W}^{(n)}$-weighted parent
    compute $w^\star$ via the AMIS denominator (5); periodically snapshot q_n into $\mathcal{S}$
end if
return $\theta^\star$ for external simulation and discrepancy evaluation

Table 1 A typical generation-staged ABC-SMC/PMC implementation (the pyABC baseline used here) versus the proposed generation-free method. The listed properties characterize the configured baseline, not ABC-SMC as a family.

Property	Classical ABC-SMC	This work
Update style	generation, barrier-synchronized	event-driven (single-arrival), no barrier
Archive	explicit, generational	reconstructed, sliding top-k
ABC likelihood	hard threshold	smooth kernel
Importance weight	against q_{t-1} only	balance heuristic over $\mathcal{S}$
State	mutable	stateless (function of history)

3.6 Relation to classical ABC-SMC

Table 1 sets the method against a typical generation-staged ABC-SMC/PMC implementation — concretely the pyABC baseline used in our experiments — property by property. Such an implementation advances a fixed population through discrete generations behind a synchronization barrier, weights each stage against the single previous proposal, and carries an explicit mutable archive; none of these is intrinsic to ABC-SMC in general (smooth kernels and richer proposals exist), but each characterizes the synchronous baseline we compare against. The proposed method replaces each of these with a barrier-free counterpart: a single-arrival update in place of the generation step, a sliding top-k archive reconstructed from history in place of the explicit population, a smooth ABC kernel in place of the hard threshold, and a balance-heuristic weight over the cumulative mixture in place of single-proposal weighting. The decisive difference is the last row: because all state is a pure function of the history, there is nothing to synchronize between workers, which is what makes the method a natural fit for the asynchronous island model of §5.

4 Theoretical Analysis

We state consistency and a CLT for the *idealized* full-mixture streaming estimator, inheriting the consistency theory for AMIS-type recycling schemes [18] under the smooth-likelihood ABC framework [7]; AMIS itself was introduced by [9], whose original convergence argument is heuristic. The streaming regime (stage size one) is a degenerate case of AMIS stages. Below, $\bar{q}_n$ denotes the snapshot denominator actually evaluated and $\frac{1}{n}\sum_{j\le n} q_{\tau_j}$ the idealized full cumulative mixture; Condition 4 bridges them. The estimator is

$$\widehat{\pi}_n(\theta) = \frac{\sum_{i=1}^{n} w_i^{\mathrm{bal}}(n)\,\delta_{\theta_i}(\theta)}{\sum_{i=1}^{n} w_i^{\mathrm{bal}}(n)}, \qquad w_i^{\mathrm{bal}}(n) = \frac{\pi(\theta_i)\,K_{\epsilon_n}(\rho_i)}{\bar{q}_n(\theta_i)}. \tag{6}$$

Condition 1 (bounded proposal-density ratio)**.** *There exist $c, C > 0$ with $c \le q_\tau(\theta)/\pi(\theta) \le C$ for every generated proposal q_τ and every $\theta \in \operatorname{supp}\pi$.*

Condition 2 (kernel regularity)**.** *K_ϵ is non-negative, integrable, peak-normalized ($K_\epsilon(0) = 1$, unit peak, not unit integral, as is standard for a probabilistic-acceptance ABC kernel), and Lipschitz in ϵ for fixed ρ.*

Condition 3 (bandwidth schedule)**.** *$\{\epsilon_n\}$ is monotone non-increasing with $\epsilon_n - \epsilon_{n+1} \to 0$ and $\epsilon_n \downarrow \epsilon_\infty > 0$. For the CLT we additionally require the schedule to be fast enough that the residual bias vanishes at the Monte Carlo scale, $\sqrt{n}\,\{\pi_{\epsilon_n}(f) - \pi_{\epsilon_\infty}(f)\} \to 0$ (which monotone convergence alone does not imply; a schedule as slow as $1/\log n$ violates it).*

Condition 4 (snapshot approximation)**.** $\sup_\theta \left|\bar{q}_n(\theta) - \frac{1}{n}\sum_{j=1}^n q_{\tau_j}(\theta)\right| \to 0$ *as* $n \to \infty$.

Theorem 1 (Consistency)**.** *Under Conditions 1–4, for every continuous π_{ϵ_∞}-integrable f, $\int f\, d\widehat{\pi}_n \xrightarrow{a.s.} \int f\, d\pi_{\epsilon_\infty}$.*

Theorem 2 (CLT)**.** *Under Conditions 1–4 and a finite second moment of $w_i^{\mathrm{bal}} f(\theta_i)$, $\sqrt{n}\big(\int f\, d\widehat{\pi}_n - \int f\, d\pi_{\epsilon_\infty}\big) \xrightarrow{d} \mathcal{N}(0, \sigma_f^2)$ with σ_f^2 the AMIS asymptotic variance at the streaming limit.*

Proof sketch. Each draw is a pair (θ_i, ρ_i) with $\theta_i \sim q_{\tau_i}$ and $\rho_i \sim p(\cdot \mid \theta_i)$; the balance-heuristic weight uses the bounded kernel $K_{\epsilon_n}(\rho_i)$, so the estimator is importance sampling on the augmented space (θ, ρ) against the smooth-ABC target $\propto \pi(\theta) K_{\epsilon_n}(\rho)$ [7]. Conditions 1–2 bound the conditional weight variance; 3 makes $\pi_{\epsilon_n} \Rightarrow \pi_{\epsilon_\infty}$ (and, with its CLT clause, controls the residual bias at the $\sqrt{n}$ scale); 4 makes the evaluated denominator $\bar{q}_n$ converge to the idealized cumulative mixture. Consistency then follows from the consistency theory for AMIS-type recycling schemes [18] applied to the streaming limit (their scheme restricts adaptation so that the weighting stage decouples; we invoke it for the idealized full-mixture estimator). The CLT yields confidence intervals for posterior expectations at the idealized limit; classical SMC/ABC-PMC central-limit theory is itself well developed [5], though not routinely exposed per run in the generational implementations we compare against. Limitations are discussed in §9.

Theory versus the efficient implementation.

Theorems 1–2 characterize the *idealized* streaming estimator that reweights against the full cumulative mixture $\frac{1}{n}\sum_{j \le n} q_{\tau_j}$. The *reported* posterior evaluates $\widehat{\pi}_n$ with the bounded snapshot denominator $\bar{q}_n$ — a draw-proportional mixture of $m \le S = 20$ history-reconstructed proposals plus a defensive prior component (mass floored at $0.5/(m{+}1)$) — not the exact full mixture; Condition 4 is precisely the assumption that $\bar{q}_n$ tracks the cumulative denominator, credible once the proposal sequence has stabilized but not guaranteed. This makes two departures from the idealized object. First, the reported (and the online proposal-time) denominator uses $m \le S = 20$ reconstructed proposals rather than all n, so the retroactive pass costs $\mathcal{O}(n\,m\,k)$ with m fixed — i.e. $\mathcal{O}(nk)$ — not the $\mathcal{O}(n^2 k)$ of a literal full mixture (Appendix A). Second, Condition 1 (a two-sided density-ratio bound relative to the prior) does not hold for

a bare top-k local Gaussian proposal across the whole support; the defensive prior component and the uniform prior-draw bootstrap phase (Alg. 1) supply only partial coverage, so formalizing the estimator with a persistent defensive mixture $\delta\pi+(1-\delta)q_n$ and a complete CLT with explicit rate is left to future work. We therefore read §4 as the asymptotic target the method is designed around, and verify the calibration of the estimator *as actually implemented* directly by simulation-based calibration (§7.2).

5 Implementation in Propulate

The algorithm is a stateless Propulate propagator: tolerance schedulers are pure functions of history; the archive is the top-k history members by smallest discrepancy under the reconstructed bandwidth; the smooth-kernel mixture weights are formed in log-space for numerical stability. The reported posterior is the retroactive AMIS estimator (6), computed off the timed inference path from the saved history (a pure function of it, hence identical whether computed online or after a crash/restart); the bare proposal-time weight (5) is retained separately for the effective-sample-size diagnostic. This retroactive step is a one-time pass of cost $\mathcal{O}(nk)$ in the n evaluated particles and archive size k, run after the timed budget and therefore excluded from the throughput measurements by design (the systems comparison is over simulation throughput). For cost-bearing simulators it is negligible against the simulation time; for near-instantaneous simulators whose history reaches $\mathcal{O}(10^7)$ particles it is non-trivial but bounded ($\mathcal{O}(nk)$ with the fixed $m \leq S = 20$ snapshot count), and is evaluated in fixed-memory chunks (Appendix A) so it stays a bounded fraction of wall-clock and memory. For an apples-to-apples comparison, pyABC's `UniformAcceptor` is replaced by a probabilistic-rejection acceptor using the same $K_\epsilon(\rho)$ and bandwidth schedule as the asynchronous side. The acceptor and kernel are thus matched across methods; the synchronous baseline differs in its generation-staged *execution* (the systems variable under test) and, on the inference side, in retaining classical population weighting rather than the streaming AMIS estimator, whose quality we check separately (§7.2). Distributed execution uses Propulate's barrier-free island model over MPI; for wall-time-limited HPC experiments the synchronous baseline uses fixed populations/generations, which yields more interpretable comparisons than contrasting methods that stop under different ϵ rules.

6 Experimental Design

6.1 Research questions

(RQ1) Does the method recover posteriors comparable to established ABC baselines? **(RQ2)** Does asynchronous execution improve wall-clock efficiency and utilization under runtime heterogeneity? **(RQ3)** Does it scale on a realistic, expensive simulator, the regime where running a synchronous matched baseline would itself be prohibitively wasteful?

Table 2 Benchmark suite and role in the evaluation.

Benchmark	Role	Outputs
Gaussian mean	sanity / calibration	analytic-posterior error, SBC coverage
g-and-k	intractable-likelihood	posterior recovery, Wasserstein vs. budget
Lotka–Volterra	simulator-based ABC	posterior recovery, quality vs. time
Cellular Potts	realistic HPC workload	throughput, utilization, scaling, posterior

6.2 Benchmarks, baselines, and metrics

We use four benchmarks of increasing realism (Table 2): Gaussian-mean inference (analytic posterior; sanity and calibration), the g-and-k distribution (classical intractable-likelihood benchmark), the stochastic Lotka–Volterra system, and a Cellular Potts tissue model (`cellsInSilico`) as the realistic heterogeneous-runtime workload. Baselines are rejection ABC (small problems), the matched-kernel synchronous baseline (a probabilistic-rejection pyABC variant, the main comparator), and pyABC as an external reference. Statistical metrics: posterior mean error, (sliced) Wasserstein distance to the truth, and credible-interval coverage via simulation-based calibration (SBC). Computational metrics: simulation throughput, worker utilization / idle fraction, and wall-clock time to a fixed posterior-error target. Convergence curves are placed on a shared time grid via last-observation-carried-forward resampling so asynchronous (fine-grained) and synchronous (per-generation) records compare fairly.

6.3 HPC studies

Three dedicated studies probe the asynchrony claim: (i) *stochastic runtime heterogeneity* (a log-normal post-evaluation delay with sweep over spread σ); (ii) *straggler tolerance* (one worker given a permanent post-evaluation delay scaled by $0\times$ (control), $1\times, 5\times, 10\times, 20\times$); and (iii) *strong scaling* at fixed wall-clock budget over worker counts from 1 up to 288 (six fully-packed nodes) on Lotka–Volterra and up to 384 (eight nodes) on Cellular Potts, with up to five replicates each. A hyperparameter *sensitivity* grid (archive size, perturbation scale, tolerance scheduler, initial tolerance) and an *ablation* (hard vs. smooth kernel; with vs. without AMIS reweighting) complete the suite.

7 Results

Unless noted, results are five-replicate medians on JUWELS; figures compare the matched-kernel synchronous baseline (labelled *Synchronous*) against the proposed asynchronous method (*Asynchronous*, ours). We lead with the computational evidence (RQ2), the paper's central claim, then verify that inference quality is not sacrificed (RQ1), and close with the realistic workload (RQ3).

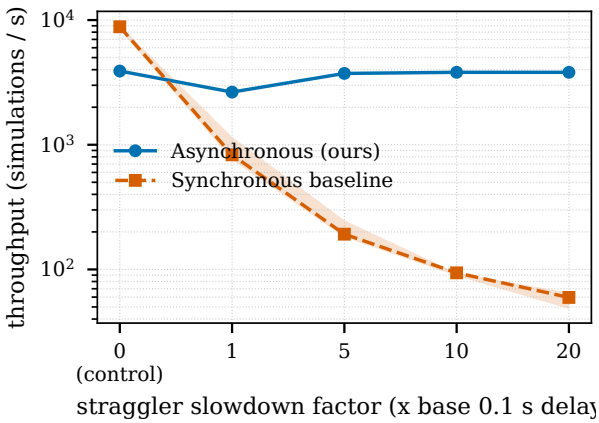

Fig. 1 Throughput (median + inter-quartile range over five replicates) versus straggler slowdown factor, including a 0× no-straggler control. One worker carries a permanent post-evaluation delay of 0.1 s scaled by the factor. The synchronous baseline degrades sharply at the generation barrier (from ≈8800 sims/s at the control to ≈60 at 20×); the asynchronous method is robust (≈3900 throughout). Note the logarithmic throughput axis.

7.1 Computational advantage (RQ2)

Straggler robustness.

This serves as a clean demonstration of the barrier-free advantage (Fig. 1). One worker is given a permanent post-evaluation delay of 0.1 s scaled by 0× (control), 1×, 5×, 10×, 20×. As the injected delay grows from the control to 20×, the synchronous baseline's throughput collapses by more than two orders of magnitude (median ≈ 8800 → 60 simulations/s) because each generation blocks on the straggler, whereas the asynchronous method routes work to available workers and holds essentially flat (≈ 3900 → 3800 simulations/s); only at the 0× control is the baseline faster, where a generation barrier is cheap for this near-instant simulator. Posterior quality is comparable throughout (final Wasserstein-to-truth ≈ 0.07–0.08 for both methods; §7.2).

Runtime heterogeneity.

Under stochastic log-normal runtime noise the asynchronous method sustains higher utilization (lower idle fraction) than the synchronous baseline, which idles at generation barriers (Fig. 2). Crucially, this utilization converts directly into inference quality: at a fixed wall-clock budget the synchronous baseline completes fewer and fewer simulations as the runtime spread σ grows (it idles at every barrier) and its posterior-mean error to the analytic Gaussian posterior climbs several-fold (from ≈0.05 to ≈0.28), whereas the asynchronous method completes several-fold more simulations and holds its error essentially flat (≈0.007–0.011) (Fig. 3). Removing the barrier therefore preserves not just the simulation rate but the *inference* delivered per unit wall-clock.

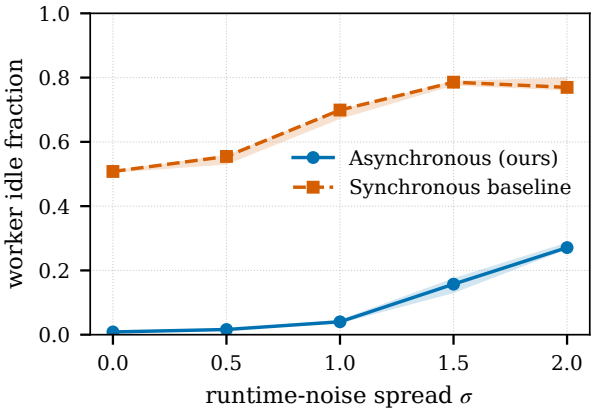

Fig. 2 Worker utilization loss under runtime heterogeneity: the fraction of worker wall-clock spent idle versus the runtime-noise spread σ (five replicates; bands are inter-quartile ranges). The synchronous baseline idles at generation barriers (its idle fraction rises from ≈0.51 to ≈0.79 as heterogeneity grows) while the asynchronous method stays near-fully utilized (idle fraction ≈0.01 rising to ≈0.27).

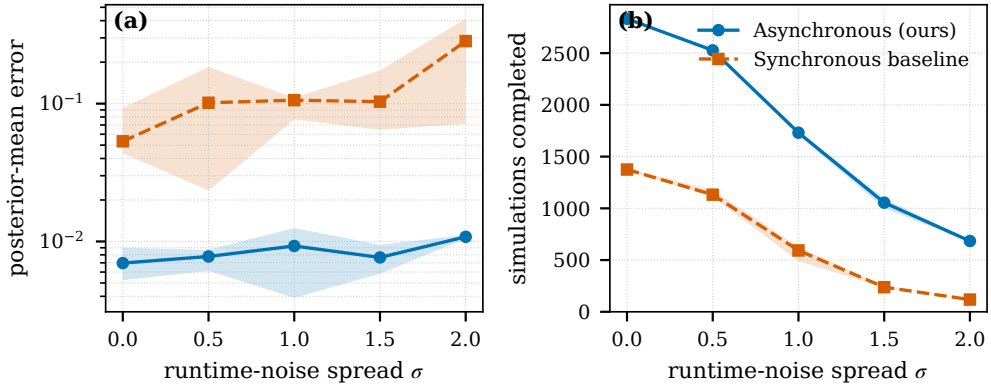

Fig. 3 Inferential efficiency under runtime heterogeneity (Gaussian-mean benchmark, analytic posterior, fixed 60 s budget, five replicates; bands are inter-quartile ranges). Left: posterior-mean error to the analytic posterior versus runtime-noise spread σ; the synchronous baseline degrades sharply while the asynchronous method stays flat. Right: simulations completed within the budget; the synchronous baseline is starved by barrier idling as σ grows, whereas the asynchronous method sustains several-fold higher completion.

Parameter-coupled runtime (every-particle bias).

The study above slows simulations independently of their parameters; a sharper test couples runtime *to the inferred parameter*, directly probing the every-particle bias of Limitation (iv). On the Gaussian-mean benchmark (48 workers, 60 s budget, five replicates) we set the post-evaluation delay to $d(\mu) = \min\big(d_0\, e^{\,c(\mu-\mu_c)/\ell},\, d_{\max}\big)$ with base $d_0 = 0.02\,\mathrm{s}$, centre $\mu_c = 0$, length scale $\ell = 0.15$, and cap $d_{\max} = 0.5\,\mathrm{s}$, and sweep the coupling strength $c \in \{0, 0.5, 1, 2\}$ ($c = 0$ recovers uniform runtime). Larger

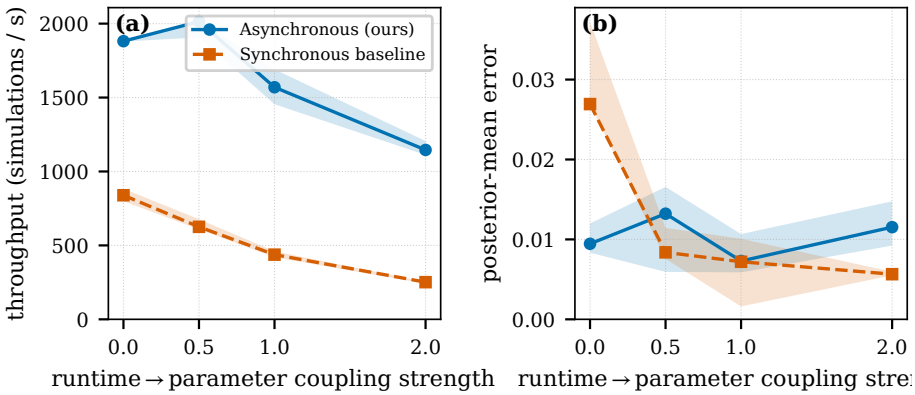

Fig. 4 Parameter-coupled runtime on the Gaussian-mean benchmark (analytic posterior; 48 workers, 60 s budget, five replicates; bands are inter-quartile ranges). The post-evaluation delay grows with the inferred mean, so one region of parameter space is sampled far more often than its mirror image. (a) Aggregate throughput falls as the coupling steepens (more steeply for the barrier-bound baseline), confirming that the fast region dominates the raw sample counts. (b) Posterior-mean error to the analytic posterior nonetheless does not increase with coupling and stays comparable between the asynchronous method and the discard-latecomers synchronous baseline, so the over-representation does not measurably distort the reported posterior at these coupling levels.

c makes one region of parameter space evaluate far faster than its mirror image, so the asynchronous "keep every accepted particle" rule over-represents the fast region in the raw archive. The coupling has the intended effect: aggregate throughput falls as it steepens, by ≈40% for the asynchronous method (≈1880 → 1150 sims/s) and more steeply for the barrier-bound baseline (≈840 → 250; Fig. 4a), confirming that the fast region dominates the raw sample counts. Yet the asynchronous posterior-mean error to the analytic Gaussian posterior does not increase as the coupling steepens; it stays ≈0.01 and comparable to the discard-latecomers synchronous baseline (Fig. 4b). At the coupling levels tested the over-representation is measurable in the raw sampling rate but does not measurably distort the reported posterior; the AMIS importance weights, which divide by the cumulative proposal density, provide an additional correction on top of this raw robustness. We discuss its uncorrected status in Limitation (iv).

Strong scaling.

Lotka–Volterra is deliberately an *adversarial* simulator for a scaling study: each evaluation takes only ≈ 2 ms, so the simulator occupies only a small fraction of each worker's wall-clock and there is essentially no per-evaluation compute to distribute. A dedicated instrumented run measures this productive fraction directly (Fig. 6): at the 48-worker throughput peak a worker spends only ≈25% of its time inside the simulator, leaving ≈75% for coordination and idle waiting, and that productive share falls to ≈8% and then ≈2% as the job spreads from one node to three to six. The mechanism is the method's single shared population: every completed evaluation is broadcast to all workers so that each can propose from the current archive, an all-to-all exchange whose per-arrival cost grows with the worker count. On a near-instantaneous simulator this exchange saturates and then actively slows the productive

Table 3 Strong scaling on Lotka–Volterra: median simulation throughput (sims/s) at a 180 s budget, over five replicates (Fig. 5, left, shows the inter-quartile spread). Asynchronous (ours, $k = 100$) vs. synchronous baseline. Multi-node points use *fully packed* nodes (worker counts that are exact multiples of 48); at counts above 100 the synchronous baseline is given *population size equal to the worker count* so it can occupy every core (a population of 100 leaves the surplus idle), which is why it keeps scaling here. The asynchronous method peaks on one node and then declines as multi-node coordination dominates, so the baseline overtakes it beyond three nodes.

Workers	1	4	16	48	144	192	240	288
Nodes	<1	<1	<1	1	3	4	5	6
Synchronous	105	96	437	2952	3757	4322	4539	6034
Asynchronous	149	369	1983	6240	4195	2615	2088	1729

ranks. Aggregate throughput therefore rises almost linearly while the job fits on one node (149 → 369 → 1983 → 6240 sims/s at 1 → 4 → 16 → 48 workers) and then *declines* across fully packed multi-node jobs (worker counts that are exact multiples of 48): 6240 → 4195 → 2615 → 2088 → 1729 sims/s at 48 → 144 → 192 → 240 → 288 workers, i.e. 1 → 3 → 4 → 5 → 6 nodes, because adding ranks beyond one node lengthens the all-to-all exchange faster than it adds useful work. This is a property of coordinating a near-instantaneous simulator across a single shared population, not of asynchrony as such (precisely the regime in which asynchrony has least to offer), and it lifts once the simulator carries real per-evaluation cost that amortizes the coordination (the Cellular Potts result of §7.3; Fig. 5, right). To keep the comparison fair at high core counts we give the synchronous ABC-SMC baseline a *population size equal to the worker count* (a smaller population cannot occupy every core); it then uses all allocated cores and scales steadily, from ≈440 sims/s at 16 workers to ≈6000 at 288 (Table 3). On this free simulator the asynchronous method leads while the job stays within a few nodes (≈4.5× at 16 workers, ≈2× at one full node), but its lead *erodes* with scale and the fair synchronous baseline overtakes it beyond three nodes, where the growing all-to-all coordination cost dominates and the asynchronous method leaves most of the allocation idle. This is the honest boundary of the asynchronous advantage on a cheap, homogeneous simulator; it is largest exactly in the heterogeneous and straggler-prone regimes where barriers otherwise waste compute.

7.2 Posterior validity (RQ1)

Having established the systems advantage, we verify it does not come at the cost of inference quality: across calibration and matched-budget posterior recovery, the streaming estimator's inference quality is comparable to that of the matched synchronous baseline.

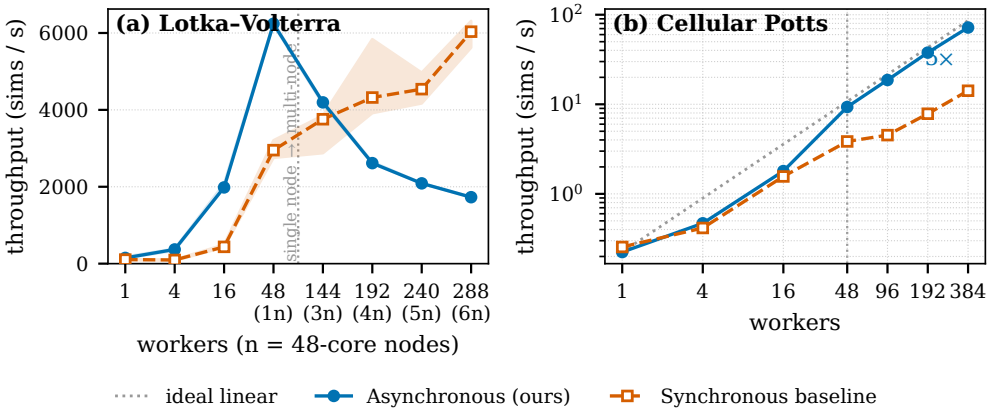

Fig. 5 Strong scaling: median simulation throughput versus worker count at a fixed wall-clock budget, asynchronous (ours) vs. a synchronous baseline given *population size equal to the worker count* so it can occupy every core. **Left:** Lotka–Volterra (near-instant simulator, ≈ 2 ms/eval; 180 s budget; categorical x-axis; bands are inter-quartile ranges over five replicates; node count n in labels). With no per-evaluation compute to distribute, the asynchronous method's single-population all-to-all coordination cost grows with the worker count: throughput rises within one node, peaks there, and then declines across multi-node jobs, so its lead over the fair, steadily scaling baseline erodes and the baseline overtakes it beyond three nodes. **Right:** Cellular Potts (costly `cellsInSilico` simulator; 1800 s budget; log–log; medians over replicates). Real per-evaluation cost amortizes the coordination, so the asynchronous method tracks ideal-linear scaling (dotted) to eight nodes (384 workers, ≈97% parallel efficiency) while the barrier-bound baseline scales only sub-linearly, a ≈5× gap at 384 workers that *grows* with scale.

Calibration.

Table 4 reports SBC empirical coverage on the Gaussian-mean model (1000 trials). The asynchronous method tracks the nominal level closely (0.50/0.78/0.87/0.92 at the 0.50/0.80/0.90/0.95 levels); the synchronous baseline under-covers at every level (0.38/0.64/0.76/0.83), i.e. it is over-confident. At 1000 trials a coverage estimate carries a Monte Carlo standard error of only ≈ ±0.007–0.016, well below the ≈0.09–0.14 gap between the methods, so the ordering is resolvable rather than noise: on this benchmark the streaming estimator is *better* calibrated than the matched synchronous baseline. It still under-covers slightly at the two highest levels (0.87 vs. 0.90 and 0.92 vs. 0.95, several binomial standard errors), so we report it as substantially better calibrated than the baseline with residual upper-level under-coverage, not as exactly nominal. The test is nonetheless limited to a one-dimensional model with an analytic posterior, not the higher-dimensional or intractable settings where the archive/proposal approximations are most likely to bite, so we read this as evidence that the streaming estimator is at least as well calibrated as the baseline, not as a guarantee of exactly nominal coverage at scale. Figure 7 shows the corresponding coverage curve and rank histogram.

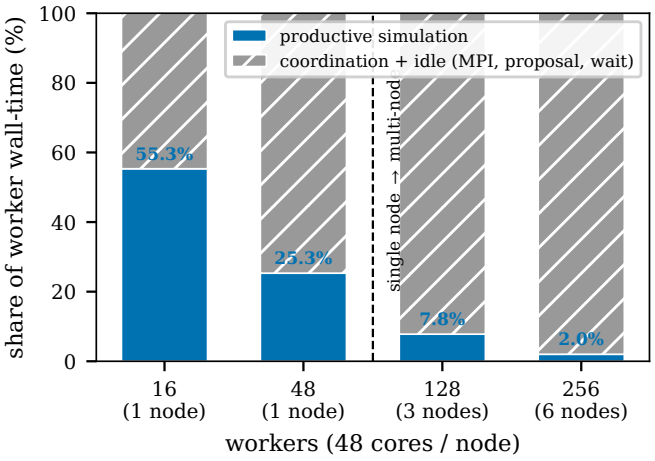

Fig. 6 Measured decomposition of asynchronous worker wall-time on Lotka–Volterra ($k = 100$; 180 s budget) from a dedicated instrumented run. The productive simulation fraction (blue, annotated) — the share of worker wall-clock spent inside the simulator — collapses from 55% at 16 workers to 2% at 256, while coordination and idle (MPI exchange, per-arrival proposal reconstruction, serialization, waiting) dominate exactly as the job spreads beyond one node (16/48 = 1 node; 128 = 3 nodes; 256 = 6 nodes). The strong-scaling decline of Fig. 5 is therefore a measured property of coordinating a near-instantaneous simulator, not an inferred one.

Table 4 SBC empirical coverage on Gaussian-mean inference (parameter μ, 1000 trials; Monte Carlo standard error $\approx \pm 0.007$–0.016). Closer to the nominal level is better.

Method	0.50	0.80	0.90	0.95
Synchronous baseline	0.38	0.64	0.76	0.83
Asynchronous (ours)	0.50	0.78	0.87	0.92

Posterior recovery.

On Lotka–Volterra the asynchronous method reaches a *lower* Wasserstein distance to the truth than the synchronous baseline at matched wall-clock budget (≈0.31 vs. ≈0.35), while on g-and-k the two are comparable (≈0.09 vs. ≈0.07) and on the expensive Cellular Potts simulator they are likewise comparable (Fig. 8); there the async−sync difference stays small (within ≈0.06 against a Wasserstein scale of ≈0.4), with an inter-quartile envelope straddling zero for most of the budget and the asynchronous method ending only marginally behind near the rejection floor (Fig. 9). Per-parameter posteriors and joint marginals are consistent with the known/reference values. In the straggler runs (which compute posterior quality) the final Wasserstein distances are comparable between methods (asynchronous ≈ 0.07–0.08 vs. synchronous ≈ 0.07). On the Gaussian-mean benchmark, which has a closed-form posterior, the asynchronous method recovers the posterior mean to within ≈ 0.002–0.038 absolute error (comparable to the synchronous baseline, ≈ 0.006–0.021, and far

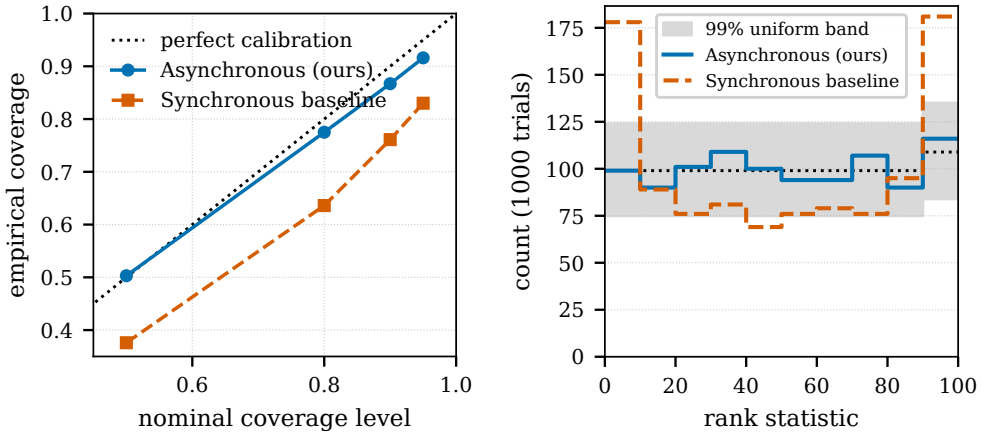

Fig. 7 Simulation-based calibration on Gaussian-mean inference: empirical coverage versus nominal level (left) and rank histogram (right). The asynchronous estimator is close to calibrated.

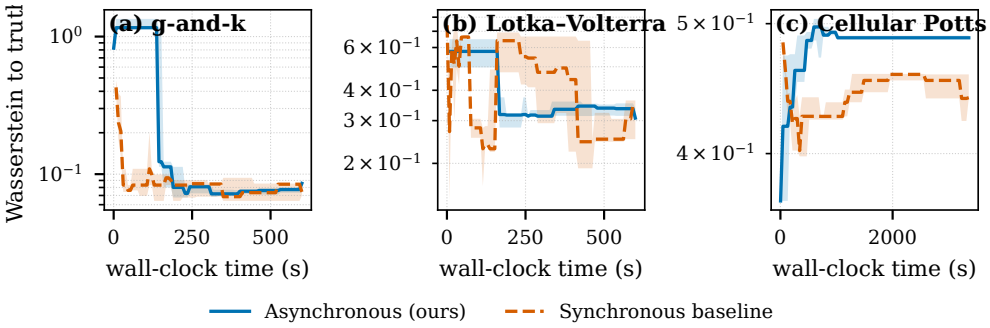

Fig. 8 Posterior quality (Wasserstein distance to the truth/reference, lower is better) versus wall-clock time on g-and-k (left), Lotka–Volterra (middle), and Cellular Potts (right), asynchronous vs. synchronous; shaded bands are inter-quartile ranges over five replicates. On Lotka–Volterra the asynchronous method reaches a lower Wasserstein distance within the budget; on g-and-k and the expensive Cellular Potts simulator the two methods are comparable.

tighter than a rejection-ABC reference) and tracks the baseline's Wasserstein-vs-time trajectory throughout the budget (Fig. 10); the SBC calibration above is unaffected.

7.3 Realistic simulator: Cellular Potts (RQ3)

The Cellular Potts benchmark runs end-to-end with the real `cellsInSilico` simulator, a realistic, expensive, heterogeneous-runtime workload and the regime the method targets.

Strong scaling.

Because each Cellular Potts evaluation carries real cost, the per-arrival coordination that capped Lotka–Volterra is amortized, and the asynchronous method scales almost

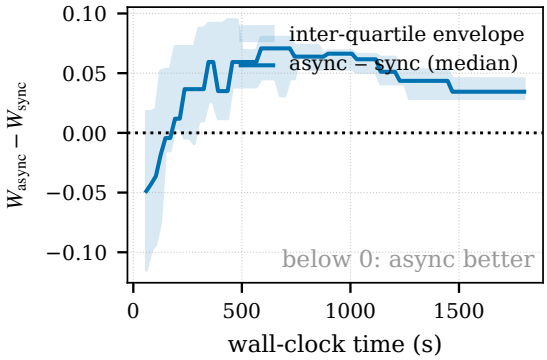

Fig. 9 Cellular Potts posterior-quality *difference* (asynchronous minus synchronous Wasserstein to the reference) versus wall-clock time, with a zero reference line and an inter-quartile envelope over five replicates (the per-method Cellular Potts curves of Fig. 8, differenced). The difference stays within ≈0.06 against an absolute Wasserstein of ≈0.4 and its envelope straddles zero for most of the budget, so the two methods are comparable on this expensive simulator; the asynchronous method ends only marginally behind, consistent with the weakly identified, near-rejection-floor regime (§7.3).

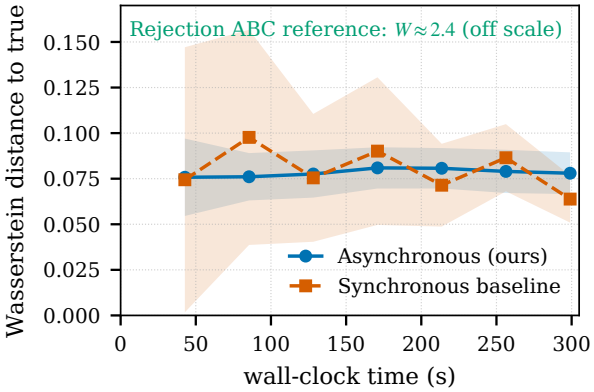

Fig. 10 Gaussian-mean posterior recovery: Wasserstein distance to the true μ versus wall-clock time, with replicate confidence bands. The asynchronous method (archive) tracks the matched synchronous baseline (generation) throughout the budget, both far below the annotated rejection-ABC reference ($W \approx 2.4$).

linearly all the way to eight nodes (Fig. 5, right): median throughput rises from 0.2 to 72.0 simulations/s over 1 to 384 workers and, unlike the near-instantaneous Lotka–Volterra case, keeps scaling *across* the single-node→multi-node boundary, each node-doubling roughly doubling throughput (9.3 → 18.7 → 37.8 → 72.0 sims/s at 48 → 96 → 192 → 384 workers), for ≈ 97% parallel efficiency relative to the single-node rate. The synchronous baseline, given a population equal to the worker count so it too can occupy every core, scales only *sub-linearly* over the same range (3.8 → 4.5 → 7.8 → 14.2 sims/s at 48 → 96 → 192 → 384 workers, ≈50% efficiency), so at 384 workers the asynchronous method sustains ≈5× its throughput, a gap that *widens* with scale. This is the regime the method is built for: an expensive, heterogeneous simulator where the generation barrier, not per-arrival overhead, is the binding cost.

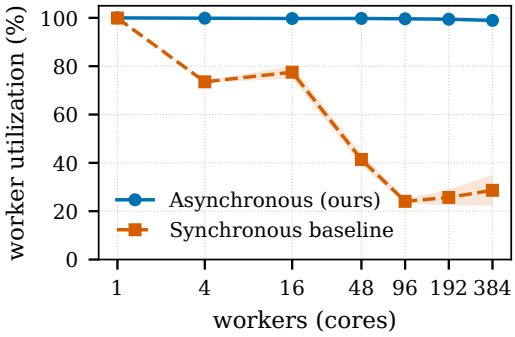

Fig. 11 Cellular Potts worker utilization (fraction of worker wall-clock spent inside the simulator) versus worker count (1800 s budget; error bars are ±1 s.d.); the synchronous baseline uses a population equal to the worker count (Fig. 5, right). The asynchronous method stays ≈98.9–99.8% utilized at every scale, while the synchronous baseline, even filling every core within a generation, then idles at the barrier waiting for the slowest simulation, so its utilization falls to ≈24–29% (41% → 24% → 26% → 29% at 48/96/192/384 workers, i.e. ≈70–76% idle at every multi-node scale). This barrier idle is the measured mechanism behind the synchronous method's sub-linear scaling in Fig. 5 (right).

Worker utilization makes the mechanism explicit (Fig. 11): the asynchronous method keeps workers ≈98.9–99.8% busy across all scales, whereas the synchronous baseline, even with every core filled within a generation, then waits for the slowest simulation at the barrier, so its utilization falls to ≈24–29% (41% → 24% → 26% → 29% from 48 to 384 workers, i.e. ≈70–76% idle at every multi-node scale). This barrier idle, not per-arrival overhead, is what holds the synchronous throughput to sub-linear scaling while the asynchronous method tracks the ideal. (We report medians throughout.)

Posterior quality.

On the inference side the asynchronous method produces a posterior comparable to the synchronous baseline (Fig. 12): the cell-motility parameter concentrates near its reference value, while the division rate is only weakly identified (for *all* methods, the baseline included). This weak identification is a structural property of the inference problem, not of asynchrony: at this scale the division-rate and motility parameters are near-degenerate (both chiefly modulate the cell count that the summaries resolve), so neither method separates them and both improve only modestly on rejection ABC. The matched comparison therefore isolates the systems advantage as *orthogonal* to identifiability: both methods are limited equally by the problem, and the asynchronous method delivers its barrier-removal advantage without sacrificing posterior quality relative to the baseline. Establishing the same quality with a synchronous matched baseline at the still larger scales the method targets would itself be prohibitively wasteful, precisely the cost the barrier-free design removes.

7.4 Ablation

Reverting the smooth kernel to a hard indicator degrades the estimator relative to the full method, while removing the AMIS cumulative-mixture reweighting is approximately neutral on this uniform-runtime analytic target (Fig. 13). On the

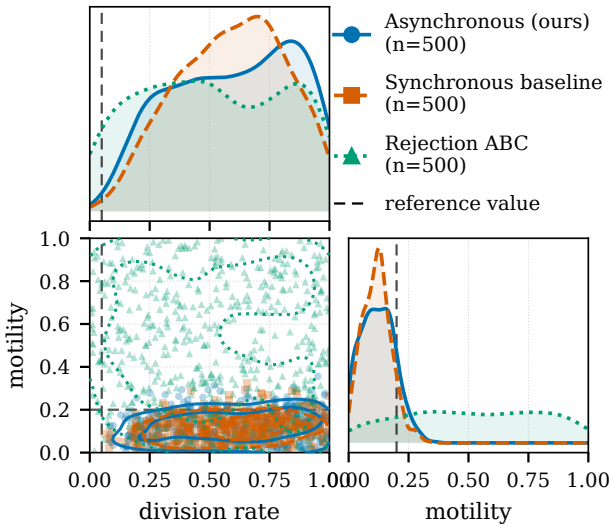

Fig. 12 Cellular Potts posterior (division rate, motility) on the real `cellsInSilico` simulator: asynchronous (blue), synchronous baseline (red), and rejection ABC (green), with dashed lines at the reference values. The asynchronous posterior is drawn from its AMIS estimator (500 resampled draws); the synchronous baseline and rejection ABC show their final populations (500 each). The asynchronous method and the synchronous baseline both place their motility mass at low values while the division rate is only weakly identified by the summaries for all methods, and both concentrate more tightly than rejection ABC.

Gaussian-mean benchmark the full method attains a final Wasserstein-to-truth of 0.073 (CI $[0.067, 0.079]$); the hard-indicator kernel raises it to 0.076, whereas dropping AMIS leaves it essentially unchanged (0.072; Fig. 13a), with the remaining hyperparameter variants scattered around the full-method reference and degrading only toward the largest perturbation scales (0.084–0.087). That AMIS is neutral here is consistent with its role: its reweighting corrects the over-representation induced by *runtime–parameter coupling* (§7.1), which this uniform-runtime target does not exercise. The AMIS-isolation trajectory (Fig. 13b) accordingly shows the full method and the no-AMIS variant tracking each other throughout the converged regime (both ≈ 0.072–0.073 after 40 s).

7.5 Sensitivity

Across the full hyperparameter grid *on the Gaussian-mean benchmark* (the analytic-posterior sanity model) the asynchronous method's posterior quality is stable, with no region of catastrophic failure (Fig. 14). The Wasserstein distance to the true parameter stays within ≈ 0.08–0.15 across the grid (each cell averaged over five replicates and the two smooth kernels); the archive size k matters only marginally, and quality degrades only mildly toward the largest perturbation scales and initial tolerances. The tolerance scheduler is inert under a smooth kernel — the asynchronous selector chooses ϵ by ESS-retention bisection regardless of the scheduler type — so we fixed it to the acceptance-rate rule rather than sweeping a variable that cannot change the smooth-kernel result.

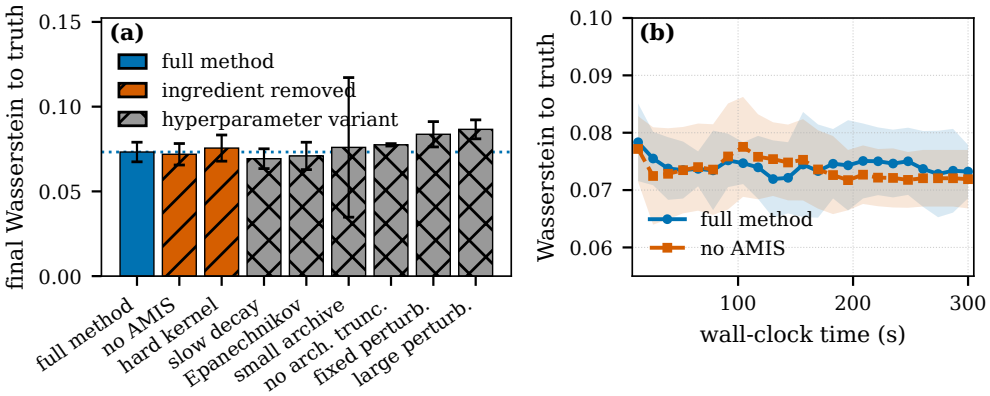

Fig. 13 Ablation on the Gaussian-mean benchmark (analytic posterior; 48 workers, 300 s budget, $k = 100$, five replicates, asynchronous method). (a) Final Wasserstein-to-truth for the full method (blue, solid), the two ingredient removals (vermillion, hatched: no AMIS, hard kernel), and hyperparameter variants (grey, cross-hatched); colour and hatch encode the class redundantly, error bars are 95% CIs, and the dotted line marks the full-method mean. (b) AMIS isolation: full method vs. no-AMIS on a shared wall-clock grid (bands are ± 1 s.d. over replicates), zoomed to the converged regime; on this uniform-runtime analytic target the two variants track each other.

We therefore did not find delicate tuning necessary to reach the quality reported above *on this benchmark*. We did not repeat the full grid on the costlier non-Gaussian benchmarks, so this robustness statement is scoped to the analytic-posterior model and to the qualitative trends observed there (scheduler-insensitive; mild, monotone sensitivity to perturbation scale and initial tolerance) rather than asserted as a general guarantee.

8 Discussion

The results support the three claims. Asynchronous execution helps most where it should: under a persistent straggler and under runtime heterogeneity, the generation barrier is the dominant cost and removing it preserves throughput that the synchronous baseline loses. For homogeneous, near-instantaneous simulators at large worker counts the asynchronous coordination overhead does not pay off: the multi-node Lotka–Volterra points, where the fair synchronous baseline overtakes the asynchronous method, make this honest boundary explicit. On the realistic Cellular Potts workload (the cost-bearing regime the design targets) the method scales almost linearly while the synchronous baseline, held at the generation barrier even when given a matching population, scales only sub-linearly, at posterior quality comparable to the baseline.

9 Limitations

We are explicit about what §4 does and does not give. (i) AMIS estimators are consistent but not finite-sample unbiased, since proposals depend on past particles; a local-decision rule in the spirit of [15] that recovers finite-sample unbiasedness for ABC

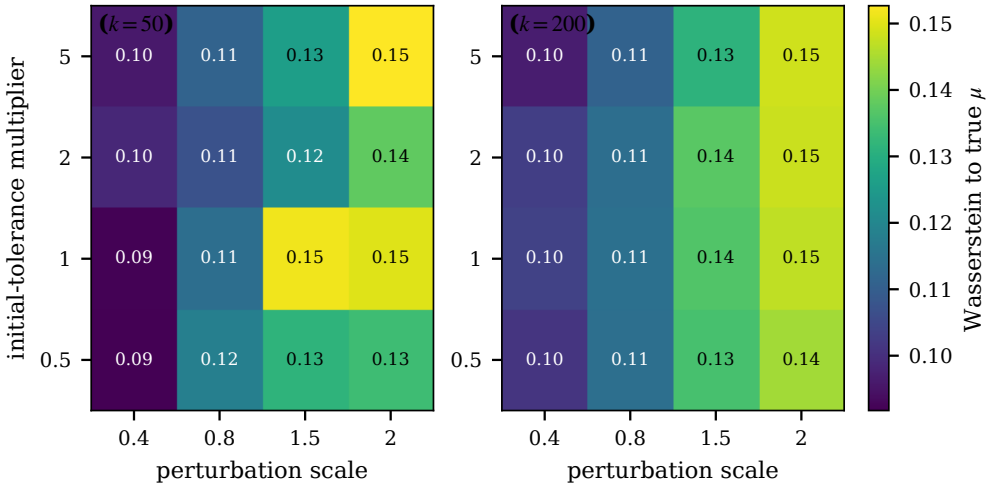

Fig. 14 Hyperparameter sensitivity of the asynchronous method, measured by posterior quality (Wasserstein distance to the true μ, lower is better; averaged over replicates and smoothing kernel). Each panel sweeps perturbation scale (x) against the initial-tolerance multiplier (y); panels are faceted by archive size k. Quality is stable across the grid (Wasserstein $\approx$ 0.08–0.15) with no failure region; the tolerance scheduler is inert under the smooth kernel and is held fixed, while larger perturbation scales degrade quality only mildly.

is future work. (ii) The bandwidth floor $\epsilon_\infty > 0$ in Condition 3 means the theorem characterizes the smooth-ABC posterior at the limit bandwidth, not the exact posterior at $\epsilon = 0$. (iii) Condition 1 is assumed for the specific archive-evolution rule, not proved. (iv) Keeping every accepted particle (unlike pyABC's discard-latecomers rule) over-represents fast-simulator parameter regions in the raw archive. The parameter-coupled-runtime experiment of §7.1 (Fig. 4) quantifies this directly: steepening the runtime→parameter coupling cuts aggregate throughput (so the fast region demonstrably dominates the raw sample counts), yet the asynchronous posterior error does not increase as the coupling steepens and stays comparable to the synchronous baseline, so the over-representation does not measurably distort the reported posterior at these coupling levels; the AMIS importance weights (which divide by the cumulative proposal density) provide an additional correction on top of this raw robustness. We therefore do not correct for it algorithmically; a residual finite-sample effect cannot be ruled out, but it is not detectable in posterior quality at the coupling levels tested. (v) The snapshot buffer is fixed at $S = 20$; adaptive sizing is a natural extension. Separately, on a methodological note for reproducibility: the post-run retroactive estimator builds its denominator from $m \leq S = 20$ history-reconstructed proposals (with a defensive prior floor), not the full evaluated history; for a near-instantaneous simulator whose history reaches many millions of particles the pass is evaluated in bounded-memory chunks, which changes memory, not the estimate.

10 Conclusion

We presented a generation-free, single-arrival-driven ABC algorithm that combines smooth-kernel ABC with streaming AMIS reweighting, is stateless and crash-recoverable, and integrates into Propulate's barrier-free island model. Its *idealized* estimator inherits AMIS's asymptotic consistency and central-limit guarantees under explicit conditions, while an optional sliding-buffer, top-k approximation used for efficient execution, which lies outside those conditions, is validated empirically by simulation-based calibration. Against a synchronous pyABC baseline matched on the ABC kernel and bandwidth schedule, the method is comparable in posterior quality to the baseline while delivering substantial HPC benefits on heterogeneous and straggler-prone workloads, including a realistic Cellular Potts simulation. Asynchronous, generation-free ABC is a promising approach for large-scale simulator-based inference on modern HPC systems.

Acknowledgements. *To be added in the final version.*

Declarations

Funding
To be added in the final version.

Conflict of interest/Competing interests
The authors declare no competing interests.

Ethics approval and consent to participate
Not applicable.

Consent for publication
Not applicable.

Data availability
The experiment outputs backing every figure and table (raw per-particle records, per-rank phase-timing logs, and the throughput/utilization/coverage summary tables), together with the resolved run configurations, will be deposited in a public repository upon publication.

Materials availability
Not applicable.

Code availability
The full implementation (the stateless Propulate propagator, the matched-kernel pyABC baseline, and the analysis and plotting scripts that regenerate every figure and table from the deposited outputs) will be released in a public repository upon publication.

Author contribution
To be added in the final version.

Appendix A Implementation Details

Tolerance schedulers.

Every scheduler is a pure function of the evaluated history and is *monotone by construction*: the propagator clips any proposed bandwidth above the history-reconstructed running minimum ϵ_{hist} (§3.2), so the effective tolerance can only decrease. Let m denote the extra-individual margin (default $m = k$). The three base rules act on the individuals *within* the current bandwidth, i.e. the hard-threshold set $\{i : \rho_i < \epsilon\}$. (This bandwidth-membership count is used *only* to drive the tolerance schedule; it is distinct from the smooth-kernel weight $K_\epsilon(\rho_i)$ that defines the proposal mixture and the reported posterior, for which every evaluated particle contributes continuously. We avoid the word "accepted" here precisely because acceptance is probabilistic under a smooth kernel.):

- **Quantile.** $\epsilon \leftarrow$ the lower-rank (non-interpolated) p-th percentile of the accepted losses, default $p = 50$; applied only once at least $k + m$ individuals are accepted.
- **Geometric decay.** $\epsilon \leftarrow \epsilon_0 \, \gamma^{\,e}$ after each completed epoch e of $k + m$ accepted individuals, default $\gamma = 0.9$.
- **Acceptance rate.** Over a sliding window of the most recent $k + m$ individuals, if the acceptance rate exceeds r_{hi} (default 0.3) tighten by a factor 0.9, otherwise hold; the rule never expands ϵ (the low-rate guard $r_{\text{lo}} = 0.1$ simply holds).

With a smooth (Gaussian or Epanechnikov) kernel the schedulers run in a *kernel-aware* mode: instead of the hard $\rho < \epsilon$ count, ϵ is chosen by bisection so that the per-step kernel-weighted effective-sample-size retention ratio meets a target (default 0.95), which keeps the bandwidth schedule comparable across kernel choices.

Proposal covariance.

The archive A_n is the top-k history members by smallest discrepancy. Component weights are formed in log-space as $\tilde{W}_j \propto w_j \, K_{\epsilon_n}(\rho_j)$ and normalised (a uniform fallback is used if all smooth weights underflow). The perturbation covariance is $\Sigma_n = s \, [\, \widehat{\text{Cov}}_{\tilde{W}}(A_n) + \lambda I \,]$ with a fixed jitter $\lambda = 10^{-6}$ for positive-definiteness and s the `perturbation_scale`; it is Cholesky-factored once per call. Each mixture component is truncated to the prior box, with its log normalising mass computed analytically so the proposal integrates to one on the support.

Matched pyABC acceptor.

For the synchronous baseline, pyABC's `UniformAcceptor` is replaced, for smooth kernels, by a probabilistic-rejection acceptor that accepts a candidate of discrepancy ρ with probability $K_\epsilon(\rho)$ (peak-normalised so $K_\epsilon(0) = 1$), using the *same* kernel K_ϵ as the asynchronous side; the hard kernel falls back to `UniformAcceptor`. The rejection step of each replicate draws from an independently seeded generator. This is the sense in which the kernel and acceptor are matched across methods (§5).

Bounded-memory posterior estimator.

The retroactive AMIS estimator (6) evaluates the snapshot denominator $\bar{q}_n$ — a draw-proportional mixture of $m \leq S = 20$ history-reconstructed proposals with a defensive prior component (mass floored at $0.5/(m{+}1)$) — at each of the n evaluated particles, at total cost $\mathcal{O}(n\,m\,k) = \mathcal{O}(nk)$ with m fixed. The m snapshots are taken draw-proportionally from the reconstructed archive (indices $0, \text{step}, 2\,\text{step}, \ldots$ with step $= \max(1, \lfloor |\text{archive}|/m \rfloor)$). For near-instantaneous simulators the history reaches $\mathcal{O}(10^7)$ particles, so the pass is accumulated in chunks of $2^{16} = 65{,}536$ history points: peak memory is $\mathcal{O}(\text{chunk} \cdot m \cdot k)$ rather than $\mathcal{O}(n)$. Each particle's denominator is evaluated independently within a single chunk (there is no log-sum-exp reduction *across* chunk boundaries), so chunking changes memory, not the value, and the result is bit-identical to the unchunked computation. Measured on the largest history (Gaussian-mean, $n \approx 3 \times 10^7$ evaluated particles, $k = 100$, $m = 20$), the one-time retroactive pass runs in ≈20 minutes on a single core with ≈0.3 GB peak memory (independent of n; the intermediate is one 2^{16}-point chunk) and parallelizes trivially across evaluation points; it runs after the timed budget and is excluded from the throughput measurements by design. For a near-instantaneous simulator this is comparable to the timed budget itself — non-trivial though bounded — whereas for cost-bearing simulators, whose histories are orders of magnitude shorter, it is negligible against the simulation time.

Appendix B Additional Experimental Protocol

Seeding.

Each experiment derives its replicate seeds deterministically from a single base seed via `make_seeds`: a `random.Random(base)` stream emits unique non-negative 31-bit integers, one per replicate, portable across platforms and independent of any global RNG state. Derived seeds (observed-data generation, the pyABC rejection step) use a structured BLAKE2b hash (`stable_seed`) of their inputs, so identical inputs reproduce identical seeds across fresh and resumed (`--extend`) runs.

Cluster and wall-clock control.

Experiments run on JUWELS (Forschungszentrum Jülich) under ParaStation MPI, 48 cores per node, scheduled with SLURM. All methods stop at a fixed wall-clock budget enforced uniformly by a monotonic-clock deadline with first-rank-hit semantics: once any rank trips the deadline it serialises its current state into the records and returns, so a partial final generation is recorded rather than discarded. Posterior estimation and plotting run off the timed inference path, so they do not count against the budget.

Benchmark configurations.

Table B1 lists the resolved settings (k is the archive/population size; the budget is per method run). The strong-scaling studies sweep the worker count from 1 to 288 (Lotka–Volterra; sub-node 1/4/16 and fully-packed multiples of 48 up to six nodes) and to 384 (Cellular Potts; up to eight nodes), with archive size $k \in \{100, 1000\}$; all other studies fix the workers shown.

Table B1 Resolved experiment configurations. Workers, per-run wall-clock budget, replicates, archive size k, and methods (A: asynchronous; S: matched-kernel synchronous baseline; R: rejection ABC).

Experiment	Workers	Budget (s)	Reps	k	Methods
Gaussian mean	48	300	5	100	A,S,R
g-and-k	48	600	5	100	A,S,R
Lotka–Volterra	48	600	5	100	A,S
Cellular Potts	48	3600	5	100	A,S,R
SBC	48	300	1000	100	A,S
Straggler	16	300	5	100	A,S
Runtime heterogeneity	48	60	5	100	A,S
Strong scaling (LV)	1–288	180	5	100/1000	A,S
Strong scaling (CPM)	1–384	1800	3–5	100	A,S
Sensitivity	48	300	5	50/200	A
Ablation	48	300	5	100	A

Benchmark-specific notes.

Gaussian mean uses $n_{\text{obs}} = 100$ observations, $\sigma = 1$, prior $\mu \sim \mathcal{U}[-5, 5]$, and a closed-form posterior used as the calibration and recovery reference. The g-and-k and Lotka–Volterra benchmarks are standard intractable-likelihood / stochastic-simulator targets. The Cellular Potts benchmark drives the real `cellsInSilico` simulator and infers the cell-motility and division-rate parameters against reference summary statistics; its per-evaluation cost dominates, which is why it uses the longest wall-clock budget.

Computing environment.

All experiments ran on the JUWELS Cluster at the Jülich Supercomputing Centre: dual-socket Intel Xeon Platinum 8168 (Skylake) nodes with 48 physical cores and ≈94 GB memory per node, connected by Mellanox EDR InfiniBand. The memory-heavy fast-simulator reruns instead used the 180 GB high-memory partition. The software stack is ParaStationMPI 5.10 (the 2025 GCC toolchain), Python 3.12.3, NumPy 1.26.4, SciPy 1.13.1, mpi4py 4.0.1, and pyABC 0.12.17, with POT for the sliced Wasserstein distance; the asynchronous method runs as a propagator in our fork of Propulate. Unless noted, ranks are placed one per physical core (`srun --ntasks-per-node=48 --cpus-per-task=1`), so a "W-worker" point occupies W physical cores packed onto $\lceil W/48 \rceil$ fully-reserved nodes; the memory-heavy Gaussian-family reruns instead placed 24 ranks per high-memory node under whole-node reservation. Each strong-scaling point reports five replicates (three to five for the largest Cellular Potts points), and figures show medians with inter-quartile ranges.

References

[1] Beaumont, M.A.: Approximate bayesian computation. Annual Review of Statistics and Its Application **6**, 379–403 (2019) https://doi.org/10.1146/annurev-statistics-030718-105212

[2] Sisson, S.A., Fan, Y., Tanaka, M.M.: Sequential monte carlo without likelihoods. Proceedings of the National Academy of Sciences **104**(6), 1760–1765 (2007) https://doi.org/10.1073/pnas.0607208104

[3] Toni, T., Welch, D., Strelkowa, N., Ipsen, A., Stumpf, M.P.H.: Approximate bayesian computation scheme for parameter inference and model selection in dynamical systems. Journal of the Royal Society Interface **6**(31), 187–202 (2009) https://doi.org/10.1098/rsif.2008.0172

[4] Beaumont, M.A., Cornuet, J.-M., Marin, J.-M., Robert, C.P.: Adaptive approximate bayesian computation. Biometrika **96**(4), 983–990 (2009) https://doi.org/10.1093/biomet/asp052

[5] Del Moral, P., Doucet, A., Jasra, A.: An adaptive sequential monte carlo method for approximate bayesian computation. Statistics and Computing **22**(5), 1009–1020 (2012) https://doi.org/10.1007/s11222-011-9271-y

[6] Taubert, O., Weißer, M., Kelle, D., Bayer, M., Knüttel, H., Comito, C., Streit, A., Götz, M.: Massively parallel genetic optimization through asynchronous propagation of populations. In: International Conference on High Performance Computing (ISC High Performance), pp. 106–124 (2023). https://doi.org/10.1007/978-3-031-32041-5_6

[7] Wilkinson, R.D.: Approximate bayesian computation (abc) gives exact results under the assumption of model error. Statistical Applications in Genetics and Molecular Biology **12**(2), 129–141 (2013) https://doi.org/10.1515/sagmb-2013-0010

[8] Fearnhead, P., Prangle, D.: Constructing summary statistics for approximate bayesian computation: semi-automatic approximate bayesian computation. Journal of the Royal Statistical Society: Series B **74**(3), 419–474 (2012) https://doi.org/10.1111/j.1467-9868.2011.01010.x

[9] Cornuet, J.-M., Marin, J.-M., Mira, A., Robert, C.P.: Adaptive multiple importance sampling. Scandinavian Journal of Statistics **39**(4), 798–812 (2012) https://doi.org/10.1111/j.1467-9469.2011.00756.x

[10] Klinger, E., Rickert, D., Hasenauer, J.: pyabc: distributed, likelihood-free inference. Bioinformatics **34**(20), 3591–3593 (2018) https://doi.org/10.1093/bioinformatics/bty361

[11] Schälte, Y., Klinger, E., Stapor, P., Hasenauer, J.: pyabc 0.12.3: high-performance distributed, likelihood-free inference. Bioinformatics **38**(13), 3354–3355 (2022) https://doi.org/10.1093/bioinformatics/btac321

[12] Drovandi, C.C., Pettitt, A.N.: Estimation of parameters for macroparasite population evolution using approximate bayesian computation. Biometrics **67**(1),

225–233 (2011) https://doi.org/10.1111/j.1541-0420.2010.01410.x

[13] Lenormand, M., Jabot, F., Deffuant, G.: Adaptive approximate bayesian computation for complex models. Computational Statistics **28**(6), 2777–2796 (2013) https://doi.org/10.1007/s00180-013-0428-3

[14] Veach, E., Guibas, L.J.: Optimally combining sampling techniques for monte carlo rendering. In: Proceedings of SIGGRAPH '95, pp. 419–428 (1995). https://doi.org/10.1145/218380.218498

[15] Paige, B., Wood, F., Doucet, A., Teh, Y.W.: Asynchronous anytime sequential monte carlo. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 27 (2014)

[16] Murray, L.M., Lee, A., Jacob, P.E.: Parallel resampling in the particle filter. Journal of Computational and Graphical Statistics **25**(3), 789–805 (2016) https://doi.org/10.1080/10618600.2015.1062015

[17] Dutta, R., Schoengens, M., Pacchiardi, L., Ummadisingu, A., Widmer, N., Künzli, P., Onnela, J.-P., Mira, A.: Abcpy: A high-performance computing perspective to approximate bayesian computation. Journal of Statistical Software **100**(7), 1–38 (2021) https://doi.org/10.18637/jss.v100.i07

[18] Marin, J.-M., Pudlo, P., Sedki, M.: Consistency of adaptive importance sampling and recycling schemes. Bernoulli **25**(3), 1977–1998 (2019) https://doi.org/10.3150/18-BEJ1042